

Domande Base di Dati

1. Scrivere l'operazione di join in algebra relazionale
2. Stimare la dimensione di un join senza eseguirlo
3. Stimare il risultato finale di un join tra due tabelle definite dal prof.
4. Utilizzo di una funzione hash per il calcolo di un join
5. Algoritmo in C per effettuare un natural join partendo da due tabelle
6. Chiusura e copertura canonica
7. Determinare se un attributo è estraneo
8. Scrivere e spiegare a cosa serve l'algoritmo che trasforma un insieme di dipendenze funzionali in una copertura canonica
9. Lossless decomposition e conservazione delle dipendenze
10. Test superchiave e dipendenze funzionali
11. Algoritmo chiusura attributo
12. Trigger
13. Hashing
14. Hashing statico e dinamico indicando le differenze di funzionamento
15. Hashing e stima della grandezza di un bucket
16. Dimensione tabella di hash
17. DB a oggetti (OODB)
18. OODB: procedura per passaggio da una relazione a una classe (persistent programming language)
19. Conversione da DB e OODB (Object Relational Mapping)
20. (Analogie e differenze tra OODBMS e ORDBMS)
21. Esempio Query con "group by" SQL e AR
22. Normalizzazione
23. (Cos'è una dipendenza funzionale)
24. Algoritmo di decomposizione BCNF
25. Algoritmo di decomposizione 3NF
26. Algoritmo di decomposizione 4NF
27. (Differenze tra BCNF, 3NF)
28. Perché la decomposizione in 3NF mantiene meglio le dipendenze funzionali rispetto alla BCNF
29. Alberi B+ e B
30. Definizione di un Albero B+ su un attributo non chiave
31. Dato un albero B+, cancellare tutti i records associati ad una chiave k
32. Dato un Albero B+, trovare tutti i records associati ad una chiave k
33. (Dato un Albero B+, inserire un nuovo valore associato alla chiave k)
34. Determinare la grandezza dell'Albero B+ necessaria a contenere l'insieme di attributi A_k della relazione R con 100^000 N-uple (formule e algoritmi)
35. Come si costruisce un albero o un indice hash su un attributo composto
36. Perché nell'algoritmo della BCNF si usa F+ e nell'algoritmo della 3NF si usa Fc
37. Algoritmo di join usando il merge sort

1) Scrivere l'operazione di join in algebra relazionale

Join Naturale

L'operatore di join naturale ($\bowtie$) dà come risultato una nuova tabella che ha come attributo l'unione degli attributi delle due tabelle di partenza. Le tuple sono scelte sulla base dell'uguaglianza dei valori degli attributi comuni alle due relazioni.

Esami

Matricola	CodCorso	Voto	Lode
29323	483	28	NO
39654	729	30	Sì
29323	913	26	NO
35467	913	30	NO

Corsi

CodCorso	Titolo	Docente	Anno
483	Analisi	Biondi	1
729	Analisi	Neri	1
913	Sistemi Informativi	Castani	2

Esami $\bowtie$ Corsi

Matricola	CodCorso	Voto	Lode	Titolo	Docente	Anno
29323	483	28	NO	Analisi	Biondi	1
39654	729	30	Sì	Analisi	Neri	1
29323	913	26	NO	Sistemi Informativi	Castani	2
35467	913	30	NO	Sistemi Informativi	Castani	2

Ogni tupla che compare nel risultato del join naturale di r_1 e r_2 , istanze rispettivamente di $R_1(X_1)$ e $R_2(X_2)$, è ottenuta come combinazione ("match") di una tupla di r_1 con una tupla di r_2 sulla base dell'uguaglianza dei valori degli attributi comuni (cioè quelli in $X_1 \cap X_2$). Inoltre, lo schema del risultato è l'unione degli schemi degli operandi

Espressione: $R_1 \bowtie R_2$		
Schema	$R_1(X_1), R_2(X_2)$	$X_1 X_2$
Istanza	r_1, r_2	$r_1 \bowtie r_2 = \{ t \mid t[X_1] \in r_1 \text{ AND } t[X_2] \in r_2 \}$
Input		Output

$r = (A, B, C, D)$

$s = (E, B, D)$

schema del risultato = (A, B, C, D, E)

$$Rr \bowtie sS = \Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B=s.B \text{ AND } r.D=s.D} (r \times s))$$

Voli

Codice	Data	Comandante
AZ427	21/07/2001	Bianchi
AZ427	23/07/2001	Rossi
TW056	21/07/2001	Smith

Linee

Codice	Partenza	Arrivo
AZ427	FCO	JFK
TW056	LAX	FCO

Prenotazioni

Codice	Data	Classe	Cliente
AZ427	21/07/2001	Economy	Anna Bini
AZ427	21/07/2001	Business	Franco Dini
AZ427	23/07/2001	Economy	Ada Cini

Voli $\bowtie$ Linee

Codice	Data	Comandante	Partenza	Arrivo
AZ427	21/07/2001	Bianchi	FCO	JFK
AZ427	23/07/2001	Rossi	FCO	JFK
TW056	21/07/2001	Smith	LAX	FCO

Voli $\bowtie$ Prenotazioni

Codice	Data	Comandante	Classe	Cliente
AZ427	21/07/2001	Bianchi	Economy	Anna Bini
AZ427	21/07/2001	Bianchi	Business	Franco Dini
AZ427	23/07/2001	Rossi	Economy	Ada Cini

Linee $\bowtie$ Prenotazioni

Codice	Partenza	Arrivo	Data	Classe	Cliente
AZ427	FCO	JFK	21/07/2001	Economy	Anna Bini
AZ427	FCO	JFK	21/07/2001	Business	Franco Dini
AZ427	FCO	JFK	23/07/2001	Economy	Ada Cini

Osservazioni: È possibile che una tupla di una delle relazioni operande non faccia match con nessuna tupla dell'altra relazione; in tal caso tale tupla viene detta "dangling". Nel caso limite è quindi possibile che il risultato del join sia vuoto; all'altro estremo è possibile che ogni tupla di r_1 si combini con ogni tupla di r_2 . Ne segue che la cardinalità del join, $|r_1 \bowtie r_2|$, è compresa tra 0 e $|r_1| * |r_2|$.

Se il join è eseguito su una superchiave di $R_1(X_1)$, allora ogni tupla di r_2 fa match con al massimo una tupla di r_1 , quindi $|r_1 \bowtie r_2| \leq |r_2|$

Se $X_1 \cap X_2$ è la chiave primaria di $R_1(X_1)$ e foreign key in $R_2(X_2)$ (e quindi c'è un vincolo di integrità referenziale) allora $|r_1 \bowtie r_2| = |r_2|$

Join naturale e intersezione

Quando le due relazioni hanno lo stesso schema ($X_1 = X_2$) allora due tuple fanno match se e solo se hanno lo stesso valore per tutti gli attributi, ovvero sono identiche, per cui:

Se $X_1 = X_2$ il join naturale equivale all'intersezione ($\cap$) delle due relazioni

VoliCharter	Codice	Data	VoliNoSmoking	Codice	Data
	XY123	21/07/2001		SC278	28/07/2001
	SC278	28/07/2001		SC315	30/07/2001
	XX338	18/08/2001			

VoliCharter $\bowtie$ VoliNoSmoking		Codice	Data
		SC278	28/07/2001

Join naturale e prodotto Cartesiano

Viceversa, quando non ci sono attributi in comune ($X_1 \cap X_2 = \emptyset$), allora due tuple fanno sempre match, per cui:

Se $X_1 \cap X_2 = \emptyset$ il join naturale equivale al prodotto Cartesiano $R_1 \times R_2$

Si noti che in questo caso, a differenza del caso matematico, il prodotto Cartesiano non è ordinato

VoliCharter	Codice	Data	VoliCharter $\bowtie$ VoliNoSmoking	Codice	Data	Numero	Giorno
	XY123	21/07/2001		XY123	21/07/2001	SC278	28/07/2001
	SC278	28/07/2001		SC278	28/07/2001	SC278	28/07/2001
	XX338	18/08/2001		XX338	18/08/2001	SC278	28/07/2001
VoliNoSmoking	Numero	Giorno		XY123	21/07/2001	SC315	30/07/2001
	SC278	28/07/2001		SC278	28/07/2001	SC315	30/07/2001
	SC315	30/07/2001		XX338	18/08/2001	SC315	30/07/2001

2) Stimare la dimensione di un join senza eseguirlo

Per effettuare una stima ragionevole occorre possedere una stima della dimensione (numero di tuple) del file risultante dall'operazione di join. Di solito tale stima viene posta uguale al rapporto tra la dimensione del file risultante dal join e la dimensione del file prodotto cartesiano, nel caso in cui entrambe le operazioni vengano applicate sui medesimi file di input. Tale rapporto viene chiamato **selettività del join (j_s)**.

Denotando il numero di tuple di una relazione R con $|R|$ si ottiene :

$$j_s = \frac{|(R \bowtie_c S)|}{|(R \times S)|} = \frac{|(R \bowtie_c S)|}{(|R| \cdot |S|)}$$

Se non vi è alcuna condizione di $\bowtie_c$, allora $j_s = 1$ e il join è uguale al prodotto cartesiano.

Se invece nessuna tupla delle relazioni soddisfa la condizione di join, allora $j_s = 0$.

In generale, quindi, $0 \leq j_s \leq 1$. Per una operazione di join in cui $\bowtie_c$ è un confronto di uguaglianze $R.A = S.B$, otteniamo i due seguenti casi:

1. se A è una chiave per R, allora $(R \bowtie_c S) \leq |S|$, sicchè $j_s \leq \left(\frac{1}{|R|}\right)$
2. se B è una chiave per S, allora $(R \bowtie_c S) \leq |R|$, sicchè $j_s \leq \left(\frac{1}{|S|}\right)$

Avere una stima delle selettività del join per le più comuni condizioni di join permette all'ottimizzatore, date le dimensioni dei due file in input, di stimare la dimensione del file

risultante dall'operazione di join mediante la formula $\frac{|(R \bowtie_c S)|}{(|R| \cdot |S|)}$

NOTA: A e B sono attributi con domini compatibili di R e S rispettivamente

3) Stimare il risultato finale di un join tra due tabelle definite dal prof

vedi domanda 1.

4) Utilizzo di una funzione hash per il calcolo di un join

L'algoritmo di hash join è un particolare algoritmo di join che usa una hash table per memorizzare i dati.

Classic Hash Join

Il primo passo nell'algoritmo dell'hash join classico è la preparazione, per la relazione più piccola, di un hash table contenente l'attributo di join e la sua riga. Dato che alla hash table si accede applicando una funzione hash sull'attributo di join, sarà il modo più rapido per trovare la riga corrispondente. Una volta costruita questa tabella, si scandisce l'altra relazione per trovare corrispondenze nelle sue righe attraverso la hash table.

L'algoritmo richiede che ci sia spazio sufficiente in memoria affinché la relazione più piccola possa essere salvata. Date due relazioni R ed S con $|R| < |S|$, dove $|R|$ è il numero di tuple contenute in R e $|S|$ quelle coinvolte in S, i passi dell'algoritmo sono i seguenti:

1. Per ogni tupla r di R
 - aggiungere r nella tabella
 - se la dimensione della tabella uguaglia la capacità di memoria necessaria:
 - scandire S e aggiungere all'output le tuple che hanno corrispondenza
 - resettare la tabella
2. Eseguire una scansione finale di S aggiungendo le risultanti tuple all'output

$Ris_join = \{\emptyset\}$

$A = R_1 \cap R_2$

generate_hash_index(R_2, A)

```

for each  $t_1$  in  $R_1$ 
     $h = \text{bucket\_of}(t_1, A)$ 
    for each  $t_2$  in  $h$ 
        if  $(t_1.A) == (t_2.A)$ 
             $Ris\_join = t_1 t_2 - (t_1.A)$ 
        end
    end
end

```

Grace Hash Join

Questa versione dell'algoritmo di hash join evita di dover scandire nuovamente l'intera relazione S , partizionando R e S tramite una funzione hash e memorizzando queste partizioni nel disco. L'algoritmo carica per coppie di partizioni in memoria, costruisce una hash table per la più piccola delle relazioni partizionate ed esamina l'altra relazione per ricercare corrispondenze tramite la hash table corrente.

Nel caso in cui non ci sia abbastanza spazio per una o più partizione l'algoritmo viene riapplicato in modo ricorsivo, viene applicata un'ulteriore funzione hash per suddividere le partizioni in altre più piccole, che verranno poi elaborate con la stessa procedura. Dato che la ricorsione risulta molto costosa, l'algoritmo fin dal principio cerca di ridurre le probabilità che si verifichi, formando il maggior numero di partizioni durante la fase iniziale.

5) Algoritmo in C per effettuare un natural join partendo da due tabelle

NATURAL JOIN: è un tipo di operazione che ci permette di correlare due o più tabelle sulla base di valori uguali in attributi contenenti lo stesso tipo di dati

Il collegamento tra 2 tabelle si effettua attraverso la foreign-key ("f") di A e la primary-key ("p") di B ; di fatto si tratta di un prodotto cartesiano in cui si confrontano solo i record per cui

A.f = B.p

Algoritmo per effettuare join in C (due tabelle)

```
k = 0;
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++){
        if(A[i].f == B[j].p){
            C[k] = A[i] U B[j] - B[j].p // la sottrazione mi
            k++;                          // garantisce di non
        }                                // prendere due volte la
    }                                    // stessa colonna
}
```

Algoritmo per effettuare join in algebra relazionale (due tabelle)

$r = (A, B, C, D)$

$s = (E, B, D)$

schema del risultato = (A,B,C,D,E)

$$Rr \bowtie s S = \Pi_{r.A, r.B, r.C, r.D, s.E}(\sigma_{r.B=s.B \text{ AND } r.D=s.D}(r \times s))$$

$Ris_join = \{\emptyset\}$

$A = R_1 R_2$

```
for each  $t_1$  in  $R_1$ 
    for each  $t_2$  in  $R_2$ 
        if  $(t_1 \cdot A) = (t_2 \cdot A)$ 
             $Ris\_join = t_1 t_2 - (t_1 \cdot A)$ 
        end
    end
end
```

Il join esegue ($n \cdot m$) confronti pertanto la complessità e il costo sono quadratici. Un modo per rendere questo tipo di join più efficiente è quello di usare gli indici lineari:

utilizzo la chiave primaria per accedere al bucket contenente i puntatori alle chiavi esterne; posso quindi confrontare le chiavi B.p solo con le chiavi esterne di A.f contenute nel bucket.

Il costo finale sarà di conseguenza lineare.

Algoritmo per effettuare join in C (due tabelle - tre tabelle)

$itoj[n_colonne_tab1] \leftarrow$ inizializzo tutto a -1

$jtoi[n_colonne_tab2] \leftarrow$ inizializzo tutto a -1

```

x = 0;
lcom = 0;

for(int i=0; i<n_colonne_tab1; i++){
    for(int j=0; j<n_colonne_tab2; j++){
        if(tab1[i] == tab2[j]){
            lcom++;
            itoj[i] = j;
            jtoi[j] = i;
        }
        else{
            itoj[i] = -1;
            jtoi[j] = -1;
        }
    }
}

for(int i=0; i<n_righe_tab1; i++){
    for(int j=0; j<n_righe_tab2; j++){
        join = true;
        for(int h=0; h<n_colonne_tab1; h++){
            if(itoj[h] != -1){
                if(tab1[i][h] != tab2[j][itoj[h]]){
                    join = false;
                }
                tab3[x][h] = tab1[i][h];
            }
        }
        if(join == true){
            n = 0;
            for(int i=0; i<n_righe_ta1; i++){
                if(join == true){
                    tab3[x][n_colonne_tab1 + n] = tab2[j][i];
                    n++;
                }
            }
            x++;
        }
    }
}
}

```

6) Chiusura e copertura canonica

Chiusura di un insieme di dipendenze funzionali

Dato un insieme F di dipendenze funzionali, vi possono essere altre dipendenze funzionali logicamente implicate da F . Ad esempio se $A \rightarrow B$ e $B \rightarrow C$ allora possiamo dedurre che $A \rightarrow C$.

L'insieme F^+ di tutte le dipendenze funzionali logicamente implicate da F è detto **chiusura di F** .

Possiamo trovare tutti gli elementi di F^+ applicando gli assiomi di Armstrong:

- se $b \subseteq a$, allora $a \rightarrow b$ (riflessività)
- se $a \rightarrow b$, allora $k \cdot a \rightarrow k \cdot b$ (arricchimento)
- se $a \rightarrow b$ e $b \rightarrow c$ allora $a \rightarrow c$ (transitività)

Queste regole sono:

- **corrette**: generano solo dipendenze valide in R;
- **complete**: applicando queste regole si deducono tutte le dipendenze che si possono implicare da F.

Altre regole che si ottengono dagli assiomi di Armstrong:

- se $a \rightarrow b$ e $a \rightarrow c$, allora $a \rightarrow b \cdot c$ (unione)
- se $a \rightarrow b \cdot c$, allora $a \rightarrow b$ e $a \rightarrow c$ (decomposizione)
- se $a \rightarrow b$ e $k \cdot b \rightarrow c$, allora $k \cdot a \rightarrow c$ (pseudotransitività)

Algoritmo 1 per trovare F^+ , chiusura di F

$F^+ = F$

repeat

for each dipendenza funzionale f in F^+

 applica riflessività e arricchimento a f e

 aggiungi ad F^+ le dipendenze ottenute

for each coppia di dipendenze f_1 e f_2 in F^+

if (f_1 e f_2 possono essere combinate utilizzando la transitività)

then aggiungi ad F^+ le dip. ottenute

until F^+ non cambia

Chiusura di un insieme di attributi

Tipicamente i progettisti prima specificano l'insieme F delle dipendenze funzionali che possono essere facilmente dedotte dalla semantica degli attributi di R; poi usano gli assiomi per dedurre altre dipendenze funzionali valide.

Un algoritmo più efficiente consiste nel determinare ogni insieme X di attributi che appare come parte sinistra di qualche dipendenza funzionale F, e poi determinare l'insieme degli attributi dipendenti da X. Perciò per ogni insieme X di attributi, si calcola l'insieme X^+ di attributi che sono determinati funzionalmente da X sulla base di F.

X^+ è detto **chiusura** di X rispetto a F.

Dato un insieme di attributi si definisce la **chiusura di X** rispetto a F (denotata con X^+) l'insieme di tutti gli attributi che sono funzionalmente determinati da attributi in X utilizzando le dipendenze in F.

Algoritmo 2 per trovare F^+

(X è l'insieme di tutti gli attributi di R)

Per ogni $\alpha \subseteq X$, si calcola la chiusura α^+ , e per ogni $Y \subseteq \alpha^+$ generiamo la dipendenza $\alpha \rightarrow Y$.

Algoritmo per trovare α^+

$\alpha^+ := \alpha$;

while (ci sono cambiamenti in α^+) **do**

```

for each  $\beta \rightarrow \gamma$  in  $F$  do
  begin
    if  $\beta \subseteq \alpha^+$  then  $\alpha^+ := \alpha^+ \cup \gamma$ ;
  end

```

Copertura canonica

Un insieme F di dipendenze funzionali può contenere dipendenze ridondanti, ovvero che possono essere ottenute dalle altre dipendenze di F . Anche gli attributi di una dipendenza funzionale potrebbero essere ridondanti.

Intuitivamente, una copertura canonica di F è un insieme “minimale” di dipendenze funzionali equivalente a F e privo di dipendenze e attributi ridondanti.

Una **copertura canonica di F** è un insieme di dipendenze funzionali F_c tale che:

- F implica logicamente tutte le dipendenze in F_c e
- F_c implica logicamente tutte le dipendenze in F e
- nessuna dipendenza di F_c contiene attributi estranei e
- tutte le parti sinistre delle dipendenze funzionali sono uniche in F_c .

Algoritmo per calcolare copertura canonica

repeat

1. usare la regola unione per rimpiazzare le dipendenze di F $\alpha_1 \rightarrow \beta_1$ e $\alpha_1 \rightarrow \beta_2$ con $\alpha_1 \rightarrow \beta_1 \beta_2$;
2. cercare una dipendenza $\alpha \rightarrow \beta$ con un attributo estraneo in α o in β e **if** (viene trovato un attributo estraneo) **then** cancellarlo da $\alpha \rightarrow \beta$;

until F non cambia

7) Determinare se un attributo è estraneo

Consideriamo un insieme F di dipendenze funzionali e la dipendenza $\alpha \rightarrow \beta$ in F

- Un attributo A è estraneo in α se $A \in \alpha$ e F implica logicamente l'insieme di dipendenze $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$;
- Un attributo A è estraneo in β se $A \in \beta$ e l'insieme di dipendenze $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ implica logicamente F .

Verificare se un attributo è estraneo

Consideriamo un insieme F di dipendenze e una dipendenza funzionale $\alpha \rightarrow \beta$ in F .

Per verificare se $A \in \alpha$ è estraneo in α

1. calcoliamo la chiusura $(\{\alpha\} - A)^+$ rispetto a F ;
2. verifichiamo se $(\{\alpha\} - A)^+$ contiene β ; se lo contiene, A è estraneo.

Per verificare se $A \in \beta$ è estraneo in β

1. calcoliamo la chiusura α^+ rispetto alle sole dipendenze in $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$;
2. verifichiamo se α^+ contiene A; se lo contiene, A è estraneo.

8) Scrivere e spiegare a cosa serve l'algoritmo che trasforma un insieme di dipendenze funzionali in una copertura canonica

Tale algoritmo permette di ottenere un insieme di dipendenze funzionali equivalenti a quello originario ma privo di ridondanze, ossia di dipendenze funzionali che implicano logicamente altre dipendenze funzionali.

9) Lossless decomposition e conservazione delle dipendenze

Lossless-join decomposition (slide 7.38)

La decomposizione di una relazione R in due relazioni R1 e R2 è senza perdita rispetto a un insieme F di dipendenze funzionali se, per ogni istanza r che soddisfa F, si ha che:

$$r = \Pi_{R1}(r) \bowtie \Pi_{R2}(r)$$

Il join naturale deve essere eseguito su una superchiave di uno dei due sottoschemi.

Condizione necessaria e sufficiente per una Lossless-join decomposition

Teorema sulle slide:

La decomposizione di R in R1 e R2 è senza perdita rispetto a F se e solo se F^+ contiene almeno una di queste dipendenze:

- $R_1 \cap R_2 \rightarrow R_1$, oppure
- $R_1 \cap R_2 \rightarrow R_2$

Conservazione delle dipendenze (slide 7.40)

La decomposizione di R in R_1 e R_2 preserva le dipendenze se:

$$(F_{R1} \cup F_{R2})^+ = F^+$$

Se si decompone R in n relazioni $R_1, R_2, \dots, R_n$ allora deve valere:

$$(F_{R1} \cup F_{R2} \cup \dots \cup F_{Rn})^+ = F^+$$

Algoritmo

Per verificare se la dipendenza funzionale $\alpha \rightarrow \beta$ è preservata in una decomposizione di R in $R_1, R_2, \dots, R_n$ facciamo il seguente test (le chiusure di attributi sono fatte rispetto a F):

result := α ;

```
while (ci sono cambiamenti in result) do  
  for each  $R_i$  della decomposizione  
     $t = (result \cap R_i)^+ \cap R_i$ ;  
    result = result  $\cup$   $t$ ;  
end
```

end

Se *result* contiene tutti gli attributi di α , allora la dipendenza funzionale $\alpha \rightarrow \beta$ è preservata.

Questo algoritmo ha un tempo di esecuzione polinomiale. Invece calcolare $(F_{R1} \cup F_{R2} \cup \dots \cup F_{Rn})^+$ e F^+ ha tempo di esecuzione esponenziale.

10) Test superchiave e dipendenze funzionali

- Test SUPERCHIAVE:
per testare se α è “superchiave”, trovo α^+ e controllo che α^+ contenga tutti gli attributi di R.
- Test DIPENDENZE FUNZIONALI:
per controllare se la dipendenza funzionale $\alpha \rightarrow \beta$ è soddisfatta basta controllare se $\beta \subseteq \alpha^+$.

11) Algoritmo chiusura attributo

Chiusura di un insieme di attributi

Tipicamente i progettisti prima specificano l'insieme F delle dipendenze funzionali che possono essere facilmente dedotte dalla semantica degli attributi di R; poi usano gli assiomi per dedurre altre dipendenze funzionali valide.

Un algoritmo più efficiente consiste nel determinare ogni insieme X di attributi che appare come parte sinistra di qualche dipendenza funzionale F, e poi determinare l'insieme degli attributi dipendenti da X. Perciò per ogni insieme X di attributi, si calcola l'insieme X^+ di attributi che sono determinati funzionalmente da X sulla base di F.

X^+ è detto **chiusura** di X rispetto a F.

Dato un insieme di attributi si definisce la **chiusura di X** rispetto a F (denotata con X^+) l'insieme di tutti gli attributi che sono funzionalmente determinati da attributi in X utilizzando le dipendenze in F.

Algoritmo 2 per trovare F^+

(X è l'insieme di tutti gli attributi di R)

Per ogni $\alpha \subseteq X$, si calcola la chiusura α^+ , e per ogni $Y \subseteq \alpha^+$ generiamo la dipendenza $\alpha \rightarrow Y$.

Algoritmo per trovare α^+

```
 $\alpha^+ := \alpha$ ;  
while (ci sono cambiamenti in  $\alpha^+$ ) do  
  for each  $\beta \rightarrow \gamma$  in F do  
    begin  
      if  $\beta \subseteq \alpha^+$  then  $\alpha^+ := \alpha^+ \cup \gamma$ ;  
    end
```

12) Trigger

Il trigger è un'istruzione eseguita automaticamente dal sistema a fronte di una modifica (un determinato evento). Esso può essere eseguito prima o dopo operazioni d'inserimento, di cancellazione o di aggiornamento dei dati; è in grado di modificare la struttura dei dati e pertanto può influire sulla normalizzazione. Per creare un meccanismo di tipo trigger bisogna specificare le condizioni con le quali il trigger viene eseguito e specificare le azioni che andrà a fare una volta che è stato invocato.

I trigger vengono utilizzati come metodi per specificare e mantenere i vincoli d'integrità in attività come:

- dati di sintesi
- replica di DB per la registrazione delle modifiche alle relazioni speciali e aventi processi separati che applicano modifiche verso una replica.

Le query di verifica (o trigger) vengono usate nell'algoritmo BCNF per garantire che le dipendenze funzionali siano rispettate. Una decomposizione preserva le dipendenze se ciascuna delle dipendenze funzionali dello schema originario coinvolge attributi che compaiono tutti insieme in uno degli schemi decomposti.

13) Hashing

Bucket: è un'unità di memoria che contiene uno o più record (dati), tipicamente è un blocco del disco.

Bucket address table: è una tabella che contiene in una colonna il risultato della funzione hash $h=f(K)$, nell'altra colonna l'indirizzo del bucket che contiene i records con la chiave K.

In un'organizzazione del file di tipo hash otteniamo il bucket del record direttamente dal suo valore della chiave di ricerca usando una funzione hash.

La **funzione hash** prende in input il valore della chiave e restituisce l'indirizzo del bucket in cui è presente il record con la chiave. Una funzione di hash ideale deve essere:

- **uniforme:** assegna a ogni bucket lo stesso numero di valori di chiave di ricerca dall'insieme di tutti i possibili valori;
- **random:** assegna gli indirizzi in modo che ogni bucket contenga lo stesso numero di record indipendentemente dai valori della chiave di ricerca;
- è definita in un intervallo definito.

$$h = f(K)$$

K è il valore della chiave, h è l'indirizzo del bucket.

Una possibile funzione hash è implementata con l'operazione resto: somma tutti i bit della rappresentazione binaria della chiave e poi calcola il resto della divisione tra la somma e il numero di bucket; il risultato serve per trovare il bucket corrispondente.

14) Hashing statico e dinamico indicando le differenze di funzionamento

Hashing statico

L'hashing statico fissa il numero di indirizzi a bucket all'inizio. Se la chiave è primaria prende come numero di indirizzi il numero di valori distinti della chiave. Se la chiave non è primaria, si fa una stima del numero di valori.

Dato che i database crescono e si rimpiccioliscono nel tempo si hanno alcuni problemi:

- se il database cresce e il numero iniziale di bucket è piccolo, bisogna gestire tanti overflow;
- se il database diminuisce, molti bucket saranno vuoti e si spreca spazio in memoria.

L'hashing può essere utilizzato per organizzare file o gestire indici. Gli indici hash sono sempre indici secondari.

La soluzione all'overflow sarebbe generare una nuova funzione di hash e ricalcolare tutti gli indirizzi, ma sarebbe troppo costoso per questo si usa l'hashing dinamico.

Come gestire il bucket overflow:

Quando la funzione di hash genera un indirizzo di un bucket già pieno di record, si ha un bucket overflow e si gestisce in due modi:

- closed hashing: il bucket overflow viene concatenato al bucket già pieno con una lista di trabocco;
- open hashing: il record viene allocato nel primo bucket libero successivo a quello pieno.

Hashing dinamico

Le tecniche di hashing dinamico permettono di modificare dinamicamente la funzione di hash per assecondare la crescita o il rimpicciolimento del database. Un'implementazione è l'**hashing estendibile** che ha i seguenti vantaggi:

- divide, aggiunge o elimina bucket quando la dimensione del DB cambia;
- la riorganizzazione coinvolge un bucket alla volta pertanto le prestazioni sono buone.

Si sceglie una funzione hash che sia:

- uniforme;
- casuale;
- limitata in un intervallo molto grande: si scelgono 32 bit così si potranno avere al massimo 2^{32} possibili buckets. Di questi 32 bit se ne usano solamente i $(0 \leq i \leq 32)$, che aumentano/diminuiscono quando il DB cresce/decrece.

Le caratteristiche della tabella degli indirizzi ai bucket sono:

- la tabella ha 2^i ingressi;
- il numero di buckets è $\leq 2^i$ perché alcune entries possono puntare allo stesso bucket;
- ad ogni bucket B_j si associa una variabile intera i_j che indica il numero di bit più significativi in comune del prefisso hash;
- il numero di entries che puntano allo stesso bucket è $2^{(i - i_j)}$;

Searching

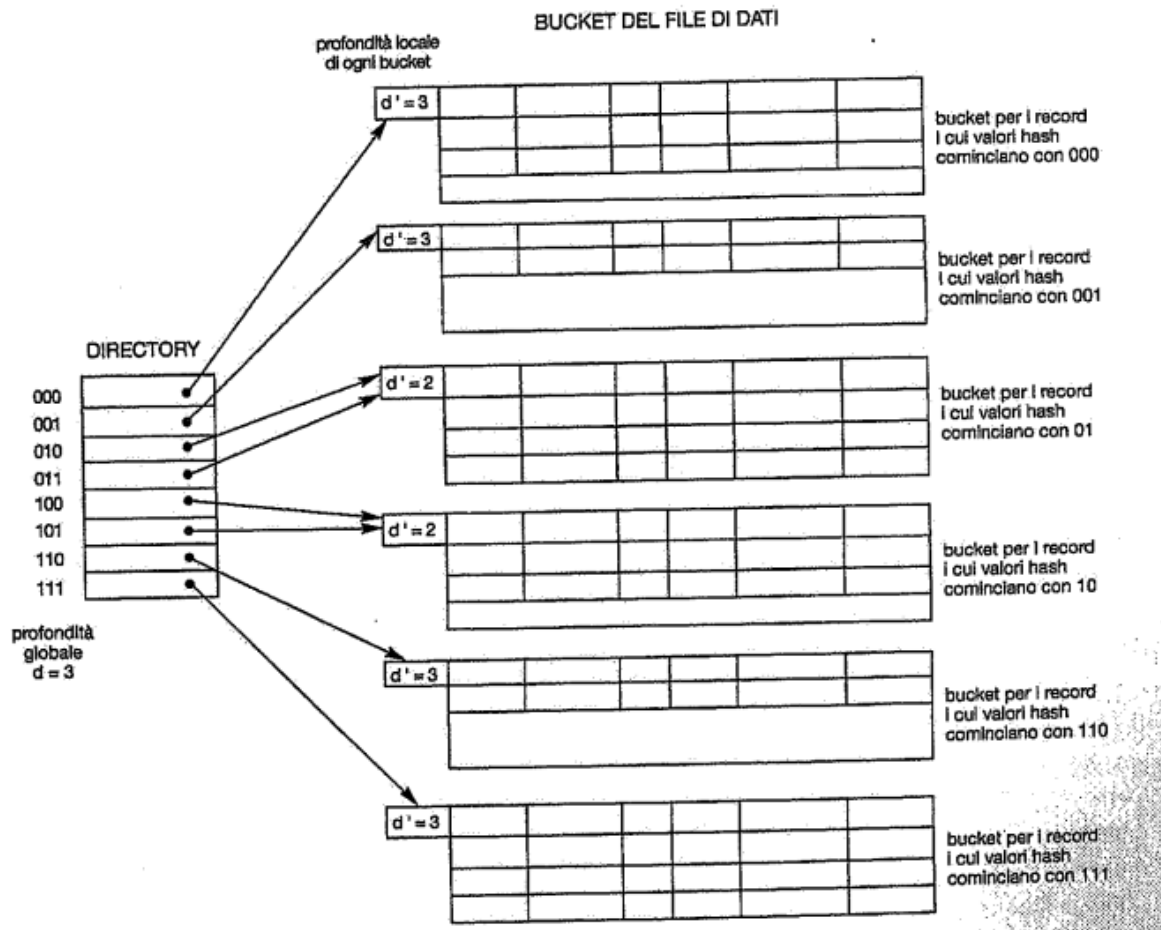
Trovare il bucket B_j che contiene la search-key K_j

1. calcolare $h(K_j) = X$;
2. usare i primi i bit di X per trovare l'indirizzo del bucket nell'address table.

Inserimento

Inserire il record a cui è associata la chiave K_j

1. Cerca il bucket B_j
2. C'è spazio nel bucket B_j ?
 - a. se c'è spazio nel bucket, si aggiunge il record
 - b. se non c'è spazio nel bucket, bisogna dividere il bucket oppure aumentare i :
 - se $i = i_j$ (una sola entry della tabella punta al bucket)
 1. si incrementa i di 1 (si raddoppia l'address table: ogni entry viene sostituita da 2 entries che contengono entrambe il valore originario). Adesso 2 entries puntano allo stesso bucket B_j ;
 2. si alloca il nuovo bucket B_z e la seconda entry si fa puntare a B_z (bisogna modificare anche i_j e i_z : $i_j = i_z = i$)
 3. ricalcola $h(K_j)$ per tutti i record del bucket B_j e li ridistribuisce tra i bucket B_j e B_z ;
 4. calcola $h(K_j)$ per il nuovo record e lo inserisce in uno dei buckets j o z (se va inserito in un bucket pieno, si aumenta i e si ripete l'operazione).
 - se $i > i_j$ (più entries puntano allo stesso bucket B_j)
 1. allochiamo un nuovo bucket B_z
 2. dato che bisogna fare spazio al bucket B_z , si modificano $i_j = i_z = i_j + 1$
 3. modifica le entries che puntavano a B_j : la prima metà non cambiano e continuano a puntare a B_j ; le restanti puntano a B_z
 4. ricalcola $h(K_j)$ per tutti i record del bucket B_j e li ridistribuisce tra B_j e B_z
 5. Prova a inserire il nuovo record.



In questa figura si ha:

- $i=3$;
- per il bucket B_1 si ha che $i_1 = 3$, infatti $2^{(i-i_1)} = 2^{(3-3)} = 1$: una sola entry punta al bucket B_1 ;
- per il bucket B_3 si ha che $i_3 = 2$, infatti $2^{(i-i_3)} = 2^{(3-2)} = 2$: due entries puntano al bucket B_2 ;

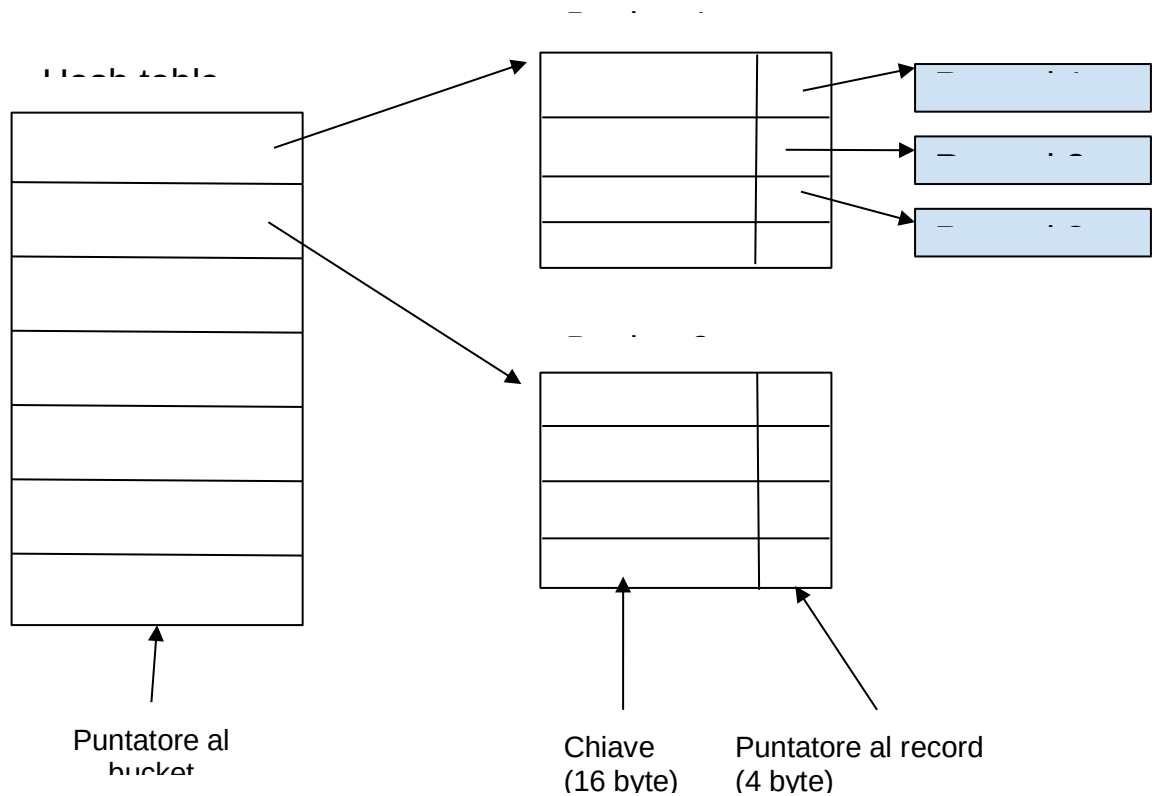
Vantaggi

- Le performance rimangono uguali anche se il DB cambia dimensioni perché modifica un solo bucket;
- l'overhead di spazio dovuto alla tabella è trascurabile perché contiene un puntatore per ogni bucket.

Svantaggi

- La tabella introduce un ulteriore livello di indirizzamento;
- la tabella delle entries può diventare molto grande (allora si usa un albero B+);
- modificare i (modificare la dimensione della tabella è un'operazione onerosa).

15) Hashing e stima della grandezza di un bucket



Sappiamo che:

- numero di chiavi $k = 100'000$
- la chiave occupa 16 byte
- i puntatori occupano 4 byte
- 1 bucket occupa un blocco di disco che è grosso 4KByte

Per trovare il numero di righe dentro un bucket dividiamo la dimensione del bucket per dimensione di una riga(chiave+puntatore al record):

$$\frac{4 \text{ KByte}}{(16 + 4) \text{ byte}} = 200 \text{ righe}$$

Dato il numero di valori di chiave calcoliamo il numero di bucket

$$\frac{100'000}{200} = 500 \text{ bucket}$$

Dato che stiamo lavorando in bit, per puntare a 500 buckets, il codice hash deve avere almeno 9 bit ($2^9 = 512$) per poter indirizzare 500 buckets:

$$\text{dimensione totale} = 512 * 4\text{KByte} = 2\text{MByte}$$

16) Dimensione tabella di hash

Tabella di hash: Una tabella hash è una struttura dati efficiente per implementare insiemi dinamici che supportino le operazioni di inserimento, cancellazione e ricerca. Tali operazioni hanno un costo computazionale costante $O(1)$.

Dati “n” record e “k” chiavi. Avremo:

- $k < n \rightarrow$ indice secondario
- $k = n \rightarrow$ indice primario

“p” indica invece il numero di entrate nella tabella di hash, ossia il numero di bucket. La funzione di hash si occupa di distribuire i valori il più uniformemente possibile

The diagram shows the formula for the dimension of a hash table: $Dimensione = p \left(\frac{k + l}{p} \right) (8 + 8) \text{ byte}$. Annotations include: 'n di chiavi di overhead' pointing to the 'l' in the numerator; '64 bit' pointing to the first '8' in the parentheses; 'Chiave 64 bit ptr' pointing to the second '8' in the parentheses; 'Entrate tabella di hash' pointing to 'p' in the denominator; and 'Entrate tabella di hash' pointing to the 'p' in the multiplier.

17) DB a oggetti (OODB)

Un database a oggetti è un sistema per la gestione dei dati che memorizza le informazioni sotto forma di oggetti con le stesse modalità di programmazione object-oriented viste per il C++, permettendo così di oltrepassare le limitazioni presenti nel modello relazionale. Gli OODB nascono per la necessità di avere tipi di dati complessi (dati multimediali, dati composti, ...); le informazioni vengono quindi salvate come oggetti.

Ciascun oggetto ha un identificatore, uno stato, e un comportamento:

- l'identificatore univoco (OID) garantisce l'individuazione in modo univoco dell'oggetto, e permette di realizzare riferimenti tra oggetti; rimane fin quando l'oggetto non viene eliminato
- lo stato è l'insieme dei valori assunti dalle proprietà dell'oggetto
 $\rightarrow$ è in generale un valore a struttura complessa
- il comportamento è descritto dall'insieme dei metodi che possono essere applicati all'oggetto

Gli oggetti sono associati ad un tipo (interfaccia) e ad una classe (implementazione)

- un tipo è una astrazione che permette di descrivere lo stato e il comportamento di un oggetto; sono così descritte le proprietà di un oggetto (la parte statica) e l'interfaccia dei suoi metodi (la parte dinamica).
- una classe descrive l'implementazione di un tipo
 $\rightarrow$ struttura dei dati e implementazione di metodi tramite programmi

Si noti che gli oggetti vengono raggruppati in collezioni (estensioni).

18) Persistent Programming Languages (procedura per passaggio da una relazione a una classe)

Nei linguaggi di programmazione classici gli oggetti sono presenti solamente quando il programma è in esecuzione. Nei linguaggi dei database, invece, gli oggetti persistono sempre. Questi linguaggi permettono di manipolare i dati direttamente dal linguaggio di programmazione senza usare SQL.

Un linguaggio di programmazione persistente è un linguaggio in cui sono presenti costrutti per maneggiare i dati persistenti. Si distinguono dai linguaggi SQL-embedded per due motivi:

1. In un linguaggio embedded il tipo di sistema dell'host è diverso dal sistema in cui vengono manipolati i dati. Il programmatore è responsabile di convertire i due sistemi. Facendo così ci sono parecchi problemi. In un PPL, invece, si ha lo stesso tipo di sistema e ci sono meno problemi;
2. Nei linguaggi embedded deve esserci una traduzione tra gli oggetti del database e il linguaggio di programmazione, mentre con PPL si vanno a gestire direttamente i dati.

Esistono diversi modi per rendere persistenti i dati:

1. Rendere persistenti le classi: è il metodo più semplice, ma non è flessibile perché è giusto che alcuni oggetti siano transienti;
2. Rendere persistenti gli oggetti dopo la creazione;
3. Rendere persistenti gli oggetti attraverso puntatori persistenti;
4. Rendere persistenti gli oggetti per raggiungibilità. Alcuni oggetti vengono dichiarati persistenti (radici) e se voglio rendere persistenti altri oggetti, allora li collego alle "radici".

19) Conversione da DB e OODB(Object relational mapping)

Vengono creati su un database già esistente e hanno il compito di collegare le tuple del database con un linguaggio di programmazione.

Nascono per semplificare la vita ai programmatori di applicazioni. Hanno il beneficio che, essendo i dati in cache, richiedono meno tempo per accedervi rispetto a prenderli direttamente dal database. Non è vantaggioso per database che richiedono molti UPDATE, dal momento che hanno una ridotta capacità riguardo la query.

Un'alternativa potrebbe essere quella di non usare un linguaggio di programmazione e di usare direttamente query più complesse in SQL. Tuttavia questa tecnica ha più vantaggi che svantaggi.

20) Analogie e differenze tra OODBMS e ORDBMS

Analogie:

- supportano dati multimediali, dati complessi, dati sotto forma di oggetti, tipi referenziati e ereditarietà.
- supportano una versione integrata di SQL.
- gli ORDBMS provano ad aggiungere le caratteristiche degli OODBMS al modello relazionale, mentre OODBMS hanno sviluppato un linguaggio di query basato su un linguaggio di query relazionali.
- forniscono al database funzionalità come il controllo della concorrenza e la protezione.

Differenze:

- l'obiettivo degli OODBMS è quello di integrare un linguaggio di programmazione come c++ o java, mentre ORDBMS non hanno questo stesso scopo.
- gli OODBMS sono più appropriati in situazioni in cui l'argomento principale è l'oggetto, mentre gli ORDBMS sono migliori per applicazioni con insiemi di dati più grandi.
- le strutture di un linguaggio di query non sono supportate efficientemente dalla maggior parte degli OODBMS; mentre per gli ORDBMS sono l'obiettivo principale.

21) Esempio Query con "group by" SQL e AR

Il costrutto "group by" ha lo scopo di applicare le funzioni di aggregazione (sum(), min(), max(), avg(), count()) su sottoinsiemi di righe:

Es.

```
select branch_name, count(distinct customer_name)
  from depositor, account
  where depositor.account_number = account.account_number
  group by branch_name
```

Gli elementi che compaiono nel select ma che non fanno parte dell'aggregazione devono comparire anche nella group by. Il risultato è una tabella ordinata in base al nome delle filiali con il numero di clienti distinti per ognuna.

In algebra relazionale l'espressione equivalente è del tipo:

$G_1, G_2, \dots, G_n \mathbin{g} F(A_1), F(A_2), \dots, F(A_n) (E)$

Dove:

- $G_1, G_2, \dots, G_n$ è la lista di attributi in base al quale raggrupparli (corrisponde al group by), può essere vuota.
- Ciascuna F_i è una funzione aggregata
- Ciascun A_i è il nome di un attributo

Riprendendo l'esempio in SQL, in AR sarebbe

$\text{branch_name } \mathbin{g} \text{ count(customer_name) (depositor } \bowtie \text{ account)}$

22) La normalizzazione

La normalizzazione è un procedimento volto all'eliminazione della **ridondanza** informativa e del rischio di **incoerenza** dal database. La ridondanza introduce parecchi problemi negli schemi relazionali:

- spreca inutilmente lo spazio in memoria;
- possono verificarsi anomalie di aggiornamento / inserimento / cancellazione di tuple;

Questo processo si fonda su un semplice criterio: se una relazione presenta più concetti tra loro indipendenti, la si decompone in relazioni più piccole, una per ogni concetto. Non è sempre applicabile in tutte le tabelle perchè si possono perdere informazioni. Dalle istanze delle relazioni decomposte, potremmo non essere in grado di ricostruire la corrispondente istanza della relazione originale.

Decomponendo uno schema di relazione R sfruttando un insieme di dipendenze funzionali F in un insieme di schemi R1, R2,..., Rn vogliamo:

1. **Decomposizione Lossless-join** (senza perdite);
2. **Minimizzare la ridondanza**: le relazioni R_i dovrebbero essere in BCNF o in 3NF;
3. **Conservare le dipendenze funzionali**, altrimenti il controllo delle violazioni delle dipendenze funzionali (dello schema originario) comporterebbe la computazione esplicita di operazioni di join (molto costose);

Esistono vari livelli di normalizzazione (*forme normali*) che certificano la qualità dello schema del database: 1NF, 2NF, 3NF, 4NF, BCNF, ecc...

23) Cos'è una dipendenza funzionale

Una dipendenza funzionale è un vincolo sulle istanze delle relazioni che mi dice se l'istanza di una relazione è valida o no.

Dato uno schema di relazione $R(A_1, A_2, \dots, A_n)$ con n attributi, prendiamo due insiemi di attributi X e Y che sono sottoinsiemi di R ($X \subseteq R$ e $Y \subseteq R$). Vale la dipendenza funzionale $X \rightarrow Y$ se e solo se per ogni istanza r di R, ogni coppia di tuple t_1 e t_2 di r aventi gli stessi valori per gli attributi in X, ha gli stessi valori per gli attributi in Y:

$$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$$

Significa che i valori della componente Y di una tupla in r dipendono dai valori della componente X, oppure che i valori della componente X determinano univocamente i valori della componente Y.

Una dipendenza funzionale $X \rightarrow Y$ è **banale** se $Y \subseteq X$. Una chiave determina funzionalmente tutti gli attributi di una relazione.

Esempio:

- $A \rightarrow B$ non è sicuramente una dipendenza funzionale perché al valore $A=3$, corrispondono due valori diversi di B ($A=3, B=4$) e ($A=3, B=7$).

A	B
3	4
1	5
3	7

- $B \rightarrow A$ “potrebbe” (vedi il N.B.) essere una dipendenza funzionale perché ad ogni valore di B corrisponde univocamente un valore di A.

N.B.: La dipendenza funzionale è una proprietà della semantica o del significato degli attributi (cioè cosa rappresentano gli attributi nel mondo reale). I progettisti usano la conoscenza della semantica degli attributi di R (come sono legati tra di loro) per trovare delle dipendenze. Le dipendenze funzionali sono una proprietà dello schema di relazione R, non di una particolare istanza r. Pertanto una DF **non può essere dedotta** da una istanza r, ma deve essere esplicitamente definita dal progettista che conosce la semantica degli attributi di R.

Per esempio, se abbiamo lo schema $R(\underline{\text{codice_fiscale}}, \text{data_di_nascita})$, il progettista dice che $\text{codice_fiscale} \rightarrow \text{data_di_nascita}$ è valida ma non vale $\text{data_di_nascita} \rightarrow \text{codice_fiscale}$.

In questa istanza r1 è evidente che $\text{data_di_nascita} \rightarrow \text{codice_fiscale}$ non vale perché alla stessa data di nascita (01/01/1980) corrispondono due codici fiscali diversi:

<u>codice fiscale</u>	data di nascita	
RSSMRA80A01D969Z	01/01/1980	Mario Rossi, nato il 01/01/1980 a Genova (GE)
BNCLCU80A41D969Z	01/01/1980	Lucia Bianchi, nata il 01/01/1980 a Genova (GE)
VRDLCU99M29D969K	29/08/1999	Luca Verdi, nato il 29/08/1999 a Genova (GE)

In questa istanza r2 abbiamo 3 date di nascita differenti. Se non sapessimo che il codice fiscale fosse un codice univoco che si ottiene da nome, cognome, data di nascita, comune di nascita, ecc... potremmo pensare che $\text{data_di_nascita} \rightarrow \text{codice_fiscale}$ sia una DF valida. Infatti per ogni data di nascita abbiamo un codice fiscale diverso. Per questo motivo è importante ricordare che le DF non si ricavano dall'istanza r di uno schema di relazione!

<u>codice fiscale</u>	data di nascita	
RSSMRA74E09D969E	09/05/1974	Mario Rossi, nato il 09/05/1974 a Genova (GE)
BNCLCU80A41D969Z	01/01/1980	Lucia Bianchi, nata il 01/01/1980 a Genova (GE)
VRDLCU99M29D969K	29/08/1999	Luca Verdi, nato il 29/08/1999 a Genova (GE)

24) Algoritmo di decomposizione BCNF

Definizione BCNF

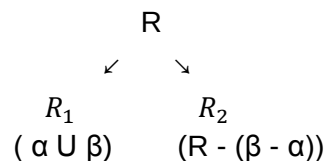
Uno schema $R(X)$ è in BCNF rispetto a un insieme F di dipendenze funzionali, se per ogni dipendenza in F^+ della forma $\alpha \rightarrow \beta$, con $\alpha \subseteq X$ e $\beta \subseteq X$, vale almeno una delle seguenti condizioni:

- $\alpha \rightarrow \beta$ è banale (ovvero $\beta \subseteq \alpha$)
- α è superchiave di $R(X)$

Esempio BCNF

R non in BCNF

$F \{ \alpha \rightarrow \beta \} \quad \alpha \subseteq R, \quad \beta \subseteq R$

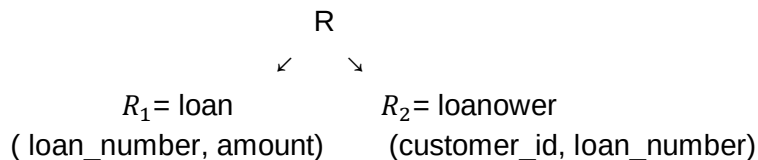


es:

R = bar_loan = (customer_id, loan_number, amount)

$F \{ \text{loan_number} \rightarrow \text{amount} \}$

// non è in BCNF in quanto
// loan_number non è SUPERCHIAVE
// e F non è banale.



Algoritmo Decomposizione in BCNF

Il senso della decomposizione è estrarre gli attributi che sono determinati da attributi non chiave e creare uno schema per ogni funzione che lega gli attributi.

result := {R};
done := false;

```

calcola  $F^+$ ;
while (not done) do
    if (esiste uno schema  $R'$  in result che non è in BCNF)
    then begin
        sia  $a \rightarrow b$  una dipendenza non banale di  $R'$  tale che  $a \rightarrow R'$  non è in  $F^+$ , e che
         $a \cap b = \emptyset$  ;
         $result = (result - R') \cup (R' - b) \cup (a, b)$ ;
    end
    else done := true;
end

```

“ $a \rightarrow R'$ non è in F^+ ” significa che a non è superchiave di R' .
 Nell’algoritmo con $(result - R')$ tolgo R' da *result* e al suo posto aggiungo due nuovi schemi:
 $(R' - b)$ e (a, b) .

25) Algoritmo di decomposizione 3NF

Definizione 3NF

Uno schema R è in 3NF rispetto a un insieme di dipendenze funzionali F se per ogni dipendenza funzionale della forma $\alpha \rightarrow \beta$ in F^+ vale almeno una delle seguenti condizioni:

- $\alpha \rightarrow \beta$ è banale (cioè, $\beta \subseteq \alpha$)
 - α è superchiave di R
 - ogni attributo A in $(\beta - \alpha)$ è contenuto in una chiave candidata di R .
- (Nota: attributi diversi possono essere contenuti in chiavi differenti)

Si noti che una relazione in BCNF è anche in 3NF.

Algoritmo Decomposizione in 3NF

Sia F_c copertura canonica di F
 $i := 0$;

```

for each dipendenza funzionale  $\alpha \rightarrow \beta$  in  $F_c$  do
    if (nessuno degli schemi  $R_j$ ,  $1 \leq j \leq i$  contiene  $\alpha, \beta$ )
    then begin
         $i := i + 1$ ;
         $R_i := \alpha, \beta$ 
    end
    if (nessuno degli schemi  $R_j$ ,  $1 \leq j \leq i$  contiene una chiave
        candidata di  $R$ )
    then begin
         $i := i + 1$ ;
         $R_i :=$  gli attributi di una chiave candidata di  $R$ 
    end
end
return  $(R_1, R_2, \dots, R_i)$ 

```

26) Algoritmo di decomposizione 4NF

Definizione 4NF

Uno schema $R(X)$ è in 4NF rispetto ad un insieme di dipendenze (funzionali o multivalore) D se per tutte le dipendenze multivalore in D^+ della forma $\alpha \twoheadrightarrow \beta$, con $\alpha \subseteq X$ e $\beta \subseteq X$, vale almeno una delle seguenti condizioni:

- $\alpha \twoheadrightarrow \beta$ è banale (cioè, $\beta \subseteq \alpha$ oppure $\alpha \cup \beta = X$)
- α è una superchiave per lo schema $R(X)$

Una relazione in 4NF è anche in BCNF

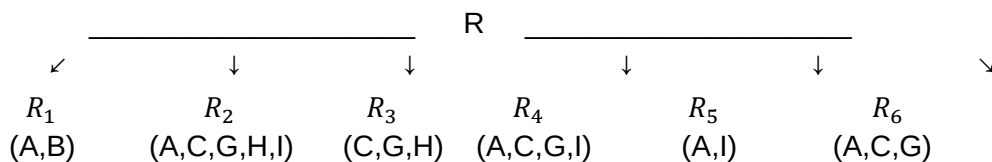
Esempio 4NF

$R = (A, B, C, G, H, I)$

$F \{ A \twoheadrightarrow B, B \twoheadrightarrow HI, CG \twoheadrightarrow H \}$

R non è in 4NF dato che $A \twoheadrightarrow B$ e A non è superchiave di R

Eseguiamo quindi la decomposizione :



R_5 ed R_6 sono presenti siccome $A \twoheadrightarrow B$ e $B \twoheadrightarrow HI$ allora $A \twoheadrightarrow HI$ e $A \twoheadrightarrow I$

Ricapitolando avremo che:

- R_1, R_3, R_5, R_6 sono in 4NF
- R_2, R_4 non sono in 4NF

Algoritmo Decomposizione in 4NF

$result := \{R\};$

$done := \text{false};$

calcola D^+ ;

Sia D' una restrizione di D^+ di R'

```

while (not done) do
  if (esiste uno schema R' in result che non sia in 4NF)
  then begin
    sia  $a \rightarrow b$  una dipendenza multivalore in  $D^+$  non banale che causa la violazione
    della 4NF su R' tale che  $\alpha \rightarrow R'$  non sia presente in  $D'$ , e  $\alpha \cap \beta = \emptyset$ . Sia:
    result := (result - R') U (R' -  $\beta$ ) U ( $\alpha, \beta$ );
  end
  else done := true;
end

```

27) Differenze tra BCNF, 3NF

Per ogni schema è sempre possibile calcolare una 3NF

- senza perdite
- che conserva le dipendenze
- introduce ridondanza
- si possono testare dipendenze funzionali senza calcolare il join

Per ogni schema è sempre possibile calcolare una BCNF

- senza perdite
- che potrebbe non preservare tutte le dipendenze

Gli obiettivi del progetto di database relazionali sono:

- BCNF
- decomposizione senza perdite
- conservare le dipendenze

Se questo non è raggiungibile, possiamo scegliere se:

- rinunciare alla conservazione di alcune dipendenze (BCNF)
- oppure ammettere la ridondanza della 3NF

28) Perché la decomposizione in 3NF mantiene meglio le dipendenze funzionali rispetto alla BCNF

Uno schema di relazione R è in 3NF se, per tutte $\alpha \rightarrow \beta$ in F^+ , almeno una delle condizioni vale:

- $\alpha \rightarrow \beta$ è banale ($\beta \subseteq \alpha$)
- α è una superchiave per R
- ogni attributo A in $\beta - \alpha$ è contenuto in una chiave candidata per R

Una relazione che è in BCNF è anche in 3NF.

Nella BCNF almeno una delle prime due condizioni deve valere.

La terza condizione è un “rilassamento” della BCNF per permettere la conservazione delle dipendenze funzionali.

29) Alberi B+ e B

Perché vengono utilizzati gli alberi B e B+?

Un indice multilivello riduce il numero di accessi ai blocchi durante la ricerca di un record. Dato che ogni livello è un file fisicamente ordinato, bisogna gestire anche inserimenti ed eliminazioni.

Per mantenere i vantaggi degli indici multilivello senza avere problemi negli inserimenti e eliminazioni, bisogna usare indici che lascino spazio per inserire nuovi blocchi. Sono chiamati indici dinamici multilivello e si implementano con alberi B e alberi B+.

Gli alberi B+ sono quelli più usati.

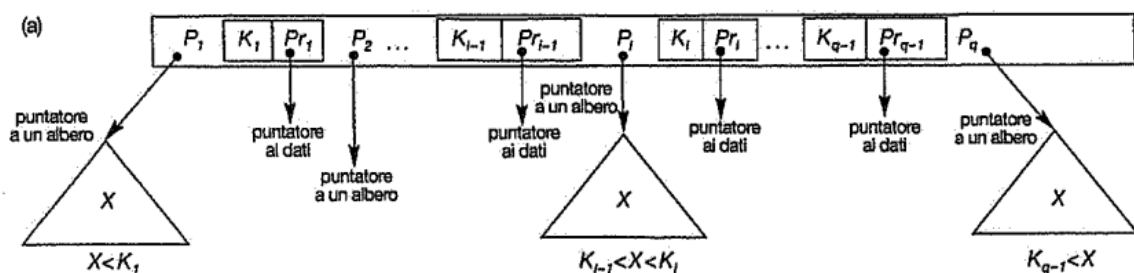
Alberi B

Un albero B soddisfa ulteriori vincoli rispetto a un normale albero di ricerca: assicura che l'albero sia sempre bilanciato e che lo spazio sprecato a seguito di una cancellazione non sia mai eccessivo.

I nodi dell'albero B sono occupati fra il 50% e il 100% e i puntatori ai blocchi di dati sono memorizzati sia nei nodi interni sia nei nodi foglia. In questo modo tutti i valori della chiave K compaiono solo una volta.

I nodi dell'albero hanno questa forma:

- ogni nodo ha questa forma $\langle P_1 \langle K_1 \ Pr_1 \rangle, P_2 \langle K_2 \ Pr_2 \rangle, \dots, P_{q-1} \langle K_{q-1} \ Pr_{q-1} \rangle, P_q \rangle$
 - P_i è un puntatore all'albero
 - Pr_i è un puntatore ai dati, cioè punta al record in cui il valore della chiave è uguale a K_i .
- Un nodo con q puntatori a un albero, contiene $q-1$ chiavi e $q-1$ puntatori ai dati;
- Deve valere $K_1 < K_2 < \dots < K_{q-1}$;
- Per tutti i valori X della chiave nel sotto albero a cui punta P_i si deve avere:
 - $X < K_i$ per $i = 1$
 - $K_{i-1} < X < K_i$ per $1 < i < q$
 - $K_{i-1} < X$ per $i = q$
- ogni nodo contiene tra $\lceil \frac{q}{2} \rceil$ e q puntatori all'albero; il nodo radice ha almeno 2 puntatori a un albero, se non è l'unico nodo; i nodi foglia hanno i puntatori a un albero tutti nulli.



Struttura nodo di un albero B con $q-1$ chiavi

Vantaggi:

- usa meno nodi dell'albero B+;

- la ricerca può finire prima di raggiungere i nodi foglia;

Svantaggi:

- Inserimento e cancellazioni sono più difficili;
- solo una piccola parte delle chiavi viene trovata prima di raggiungere i nodi foglia;
- hanno profondità maggiori rispetto agli alberi B+.

Alberi B+

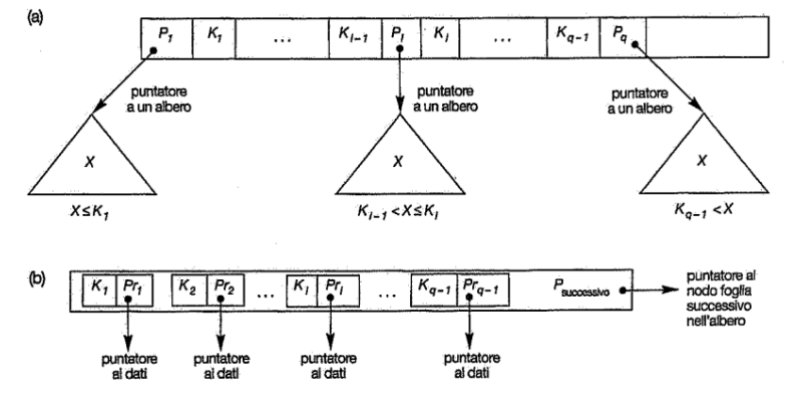
I puntatori ai dati sono presenti solo nei nodi foglia. I valori della chiave K invece possono comparire parecchie volte nell'albero (ridondanza) a diversi livelli. Grazie a queste caratteristiche gli alberi B+ solitamente hanno meno livelli.

Nodi interni:

- hanno la seguente forma: $\langle P_1K_1, P_2K_2, \dots, P_{q-1}K_{q-1}, P_q \rangle$
 - P_i è un puntatore a un albero
 - K_i è il valore della chiave
- i valori della chiave devono essere ordinati: $K_1 < K_2 < \dots < K_{q-1}$
- per tutti i valori X della chiave a cui P_i punta, si ha:
 - $X \leq K_i$ per $i = 1$
 - $K_{i-1} \leq X < K_i$ per $1 < i < q$
 - $K_i \leq X$ per $i = q$
- ogni nodo interno contiene tra $\lceil \frac{q}{2} \rceil$ e q puntatori all'albero;
- ogni nodo interno contiene q-1 valori della chiave;

Nodi foglia:

- hanno la seguente forma: $\langle K_1Pr_1, K_2Pr_2, \dots, K_iPr_i, \dots, K_{q-1}Pr_{q-1}, P_{next} \rangle$
 - Pr_i è il puntatore ai dati
 - K_i è il valore della chiave contenuta nel record a cui punta Pr_i
 - P_{next} è il puntatore al nodo foglia successivo
- i valori della chiave devono essere ordinati: $K_1 < K_2 < \dots < K_{q-1}$;
- ogni nodo foglia ha almeno $\lceil \frac{q-1}{2} \rceil$ valori;
- tutti i percorsi dal nodo radice a qualunque nodo foglia devono avere la stessa lunghezza.



30) Definizione di un Albero B+ su un attributo non chiave

La cancellazione di un record da un albero B+ (se non sono usati i bucket) diventa un problema perchè potrei impiegare molto tempo a trovare quel record. Una possibile soluzione è quella di unire a questo attributo un altro valore in modo tale da rendere unica la coppia. In alternativa, si possono usare i bucket che mi permettono di salvare dello spazio. Quest'ultimi hanno il problema dell'aggiunta di codice per poter gestire la grandezza del bucket (se non è salvato con la foglia) o se viene messo in un altro blocco sarei costretto ad eseguire più operazioni di I/O

31) Dato un albero B+,cancellare tutti i records associati ad una chiave k

- Passo 1:
Cerco il record che deve essere cancellato e lo rimuovo dal file o dal bucket (se presente)
- Passo 2:
rimuovo la coppia (chiave, puntatore) dal nodo foglia (se il bucket è diventato vuoto o se non è presente)
- Passo 3:
Se in seguito alla rimozione del valore il nodo e quello affianco hanno così pochi valori da essere uniti in un unico nodo, li unisco:
 - metto tutto in quello a sinistra e cancello l'altro nodo
 - nel padre cancello il riferimento a quel nodo e vedo se devo inserire gli altri nodi
- Passo 4:
Se dopo la cancellazione il nodo ha poche voci, ma le sue più quelle di suo fratello non possono essere unite in unico nodo, bisogna ridistribuire i puntatori:

- si distribuiscono i puntatori in modo che i due fratelli abbiano almeno il numero minimo di voci
- si aggiorna il valore della chiave di ricerca corrispondente

La cancellazione dei nodi si può propagare verso l'alto fino a quando non viene trovato un nodo con $\lceil \frac{n}{2} \rceil$ o più puntatori.

Se il nodo radice rimane con un solo puntatore, viene cancellato e il figlio diventa radice.

32) Dato un Albero B+, trovare tutti i records associati ad una chiave k

(vedi slide 12.23 - 12.24)

Ricerca dei record con chiave K

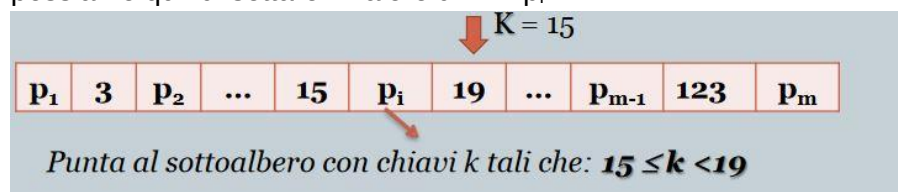
- Passo 1:

Fisso N = radice

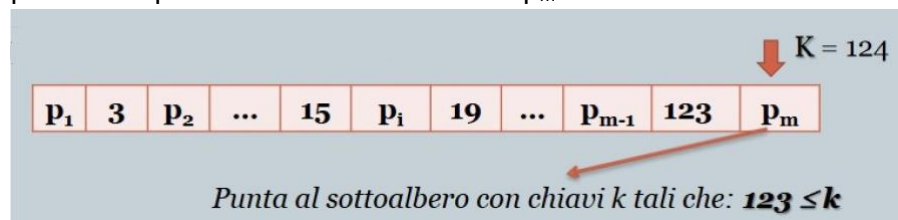
- Passo 2:

cercare nel nodo (radice) il più piccolo valore di chiave maggiore di k

se tale valore esiste (supponiamo sia K_i) seguire il puntatore p_i possiamo quindi settare il valore di $N = p_i$



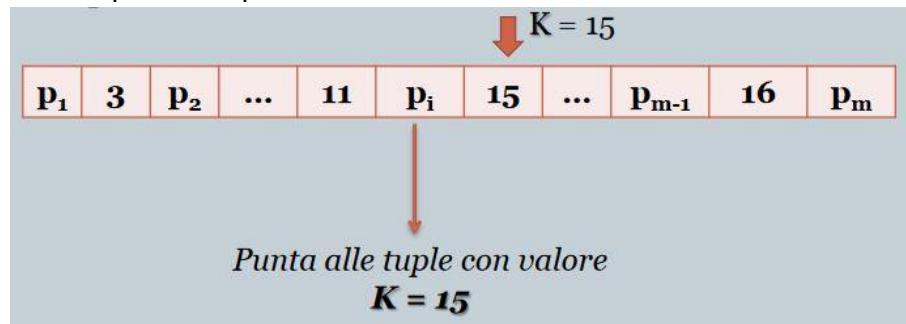
se tale valore non esiste ($k \geq K_{m-1}$) seguire il puntatore p_m possiamo quindi settare il valore di $N = p_m$



Ripetere il procedimento fin quando N non è un nodo foglia

- Passo 3:

Se il nodo raggiunto è un nodo foglia cercare il valore k ($K_i = k$) nel nodo e seguire il corrispondente puntatore verso il record o il bucket associato



- Passo 4:
Altrimenti non esiste un record con il valore k della chiave di ricerca

Osservazione:

Il costo di una ricerca nell'indice, in termini di numero di accessi alla memoria secondaria, risulta pari al numero di nodi acceduti nella ricerca. Tale numero in una struttura ad albero è pari alla profondità dell'albero, che nel B+ tree è indipendente dal percorso ed è funzione del fan-out " n " (numero di puntatori ai nodi figli di un nodo) e del numero di valori chiave K (#keyValues) presenti nell'albero:

$$prof_{B+tree} \leq \lceil \log_{\lceil \frac{n}{2} \rceil} (K) \rceil$$

33) Dato un Albero B+, inserire un nuovo valore associato alla chiave K

(vedi slide 12.25 - 12.26 - 12.27 - 12.28 - 12.29)

Inserimento del record associato alla chiave K :

- Passo 1:
cerco il nodo foglia in cui il nuovo valore deve essere posizionato
- Passo 2:
se il valore della chiave è già presente nel nodo foglia
 - aggiungo il record al file
- Passo 3:
se il valore della chiave non è presente :
 - aggiungo il record al file (creo un bucket se necessario)
 - se c'è spazio nel nodo foglia inserisco la coppia (k,p) chiave-puntatore
 - altrimenti devo sistemare l'albero
- Pass 4:
per sistemare l'albero qualora un nodo foglia fosse pieno
 - prendo le n coppie (k,p) chiave-puntatore presenti nella foglia più il nuovo nodo e li ordino.
 - metto le prime $\lceil \frac{n}{2} \rceil$ coppie nel nodo esistente e le rimanenti in un nuovo nodo. Chiamo il nuovo nodo P .
 - inserisco nel padre di P il più piccolo valore di P

- Passo 5:
qualora anche il padre fosse pieno continuiamo questo procedimento. Al più, se tutti i nodi fossero pieni, andrei ad alzare l'altezza dell'albero di 1.
- Passo 6 (*):
SPLIT a un nodo non-foglia quando si vuole inserire (k,p) in un nodo padre pieno N
 1. copiare N in un'area di memoria M che contenga n valori e (n + 1) puntatori (n ordine albero)
 2. Inserisco anche la nuova coppia (k,p) in M
 3. copia $P_1, K_1, \dots, K_{\lceil \frac{n}{2} \rceil}, P_{\lceil \frac{n}{2} \rceil}$ da M nel nodo N
 4. copia $P_{\lceil \frac{n}{2} \rceil+1}, K_{\lceil \frac{n}{2} \rceil+1}, \dots, K_n, P_{n+1}$ da M nel nuovo nodo allocato (N^A)
 5. inserisci nel nodo padre N la coppia $(K_{\lceil \frac{n}{2} \rceil}, N^A)$

* Spiegazione procedimento SPLIT di un nodo non foglia:

quando un nodo **non**-foglia è splittato, i puntatori ai figli sono divisi tra il nodo originale e il nuovo nodo. Le search-key sono trattate in maniera differente:

- quelle che stanno a destra di tutti i puntatori che si sono spostati, si spostano nel nuovo nodo;
- quelle che stanno a sinistra dei puntatori che sono rimasti nel nodo originale rimangono invariati;
- quelle che stanno tra i puntatori che si sono spostati e i puntatori che sono rimasti invariati non vengono aggiunte a nessun nodo dello split, vengono invece aggiunte al nodo padre con un puntatore che punta al nuovo nodo creato dallo split.

Se il nodo padre dovesse essere pieno si continua ricorsivamente questa procedura

34) Determinare la grandezza dell'Albero B+ necessaria a contenere l'insieme di attributi A_k della relazione R con 100°000 N-uple (formule e algoritmi)

Dati "n" record e "k" chiavi con cui costruire un albero, è necessario conoscere anche

- a = numero di figli massimo per ciascun nodo dell'albero B+
- b = numero di chiavi massimo per ciascun nodo

se $k < n$ ($\rightarrow$ indice secondario) $\rightarrow$ le foglie puntano ai bucket contenenti i record

All'ultimo livello troviamo le "k" chiavi; sapendo poi che ogni nodo contiene le "b" chiavi avremo che:

$$\frac{k}{b} \text{ nodi} \quad (\text{dove per ogni livello esistono } a^x \text{ nodi con } x[0, \dots, m])$$

Se si parte dalla radice ad ogni livello si moltiplica per il numero di nodi figlio per ciascun nodo; partendo invece dalle foglie è necessario dividere.

Avendo $\frac{k}{b}$ foglie all' m-esimo livello avremo :

$$\begin{array}{ll} \frac{1}{a} \cdot \frac{k}{b} & \text{nodi all'm-1 esimo} \\ \frac{1}{a^2} \cdot \frac{k}{b} & \text{nodi all'm-2 esimo} \\ \vdots & \\ \frac{1}{a^m} \cdot \frac{k}{b} & \text{nodi al livello 0 (o radice)} \end{array}$$

Quindi in totale ho al massimo:

$$\varphi = \sum_{i=0}^{m-1} \frac{k}{b} \left(\frac{1}{a}\right)^i = \frac{k}{b} \sum_{i=0}^{m-1} \left(\frac{1}{a}\right)^i = \frac{k}{b} \frac{1 - \left(\frac{1}{a}\right)^m}{1 - \frac{1}{a}} \quad \text{nodi in tutto l'albero}$$

dove $m = \log_a \frac{k}{b}$ numero di livelli

Esempio

Ogni nodo è fatto così:

Puntatore	Chiave	Puntatore		Puntatore
-----------	--------	-----------	------	-----------

- **b** chiavi di dimensione **d**;
- **b+1** puntatori di dimensione **8 byte**.

Quindi ogni nodo è grande **[(b+1)*8 + b*d] byte**.

L'albero è grande **numero_nodi * dimensione_nodo byte = $\varphi * [(b+1)*8 + b*d]$ byte**.

Se abbiamo:

- numero figli per nodo: $a = 4$;
- numero chiavi per nodo: $b = 5$;
- numero chiavi: $k = 100'000$.

Allora l'albero avrà:

- $\frac{k}{b} = 2000$ nodi foglia;
- $m = \log_a \frac{k}{b} = \log_4 2000 \cong 5$ livelli;

- $\varphi = \sum_{i=0}^{m-1} \frac{k}{b} \left(\frac{1}{a}\right)^i = 2000 * \sum_{i=0}^4 \left(\frac{1}{4}\right)^i = 2000 * \frac{1 - \left(\frac{1}{4}\right)^5}{1 - \frac{1}{4}} = 2664$
nodi;
- dimensione albero: $\varphi * [(5+1)*8 + 5*d]$ byte.

35) Come si costruisce un albero o un indice hash su un attributo composto

Se voglio costruire un indice hash basandomi su un insieme di attributi che non sono chiave primaria (ad esempio Nome e Peso), devo unire gli attributi in un'unica stringa. Dopodiché calcolo il codice hash sulla stringa.

Per costruire un albero unisco gli attributi in una stringa e la uso come chiave dei nodi.

MODIFICHE AL DATABASE

DELETE $r \leftarrow r - E$

INSERT $r \leftarrow r \text{ unito } E$

UPDATE $r \leftarrow \Pi_{F1, F2, \dots}(r)$

QUERY IN AR E SQL

select A1, A2, ..., An

from r1, r2, ..., rm

where P

$\Pi_{A1, A2, \dots, An}(\sigma_P(r1 \times r2 \times \dots \times rm))$

DOMANDE CLAUDIO:

Mi ha chiesto quali sono i tipi di join (natural, left outer, right outer, full outer, theta). Il theta join è quello in cui ha più condizioni. Sapere bene definizione di predicato e cosa è null per spiegarlo nei vari outer join.

Scrivere in algebra relazionale il natural join (puoi fare un es.), scrivere gli altri join in algebra relazionale -> domanda trabocchetto: non si può fare perché in algebra relazionale non esiste il concetto di null.

Scrivere algoritmo facile del join in C.

Come cambia l'algoritmo se si usano gli indici hash, quanti ne usi? Facendo un es. facile con R1(A, B, C) e R2(B, D) si usano due indici hash corrispondenti a R1.B e R2.B. quando si parla di hashing non usare il termine indirizzo (non so perché ma non gli piace) ma usa

direttamente bucket. In sostanza usando l'hashing per fare il join si fanno meno confronti, si confrontano i bucket che hanno chiavi uguali cioè l'attributo comune uguale.